

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Comparative Analysis (Low)
Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.	Bloom's Level: 4 – Analyze
2	Differentiate between clustered indexing and non-clustered indexing with suitable database examples.	Bloom's Level: 4 – Analyze
3	Compare primary indexing and secondary indexing based on storage requirements and search performance.	Bloom's Level: 4 – Analyze
4	Analyze the differences between static hashing and dynamic hashing in database systems.	Bloom's Level: 4 – Analyze
5	Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.	Bloom's Level: 4 – Analyze
6	Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.	Bloom's Level: 4 – Analyze
7	Compare dense indexing and sparse indexing with respect to memory usage and access speed.	Bloom's Level: 4 – Analyze
8	Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.	Bloom's Level: 4 – Analyze

9	Compare single-level indexing and multi-level indexing for efficient database retrieval operations.	Bloom's Level: 4 – Analyze
10	Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.	Bloom's Level: 5 – Evaluate

Code of Conduct:

I G Vyshnavi (192525260) __ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:vyshnavi

Q1. Sequential File Organization vs Heap File Organization

Feature	Sequential File Organization	Heap File Organization
Arrangement	Records are stored in a particular order, usually by key.	Records are stored in any available location without ordering.
Insertion	Slower because the correct position may need to be found.	Faster because a new record can be added to an available position.
Deletion	More difficult because maintaining the order may require rearrangement.	Easier because the record can simply be marked or removed.
Retrieval	Efficient for sequential and ordered access.	Sequential search is required for most searches.
Searching	Faster when records are ordered and binary search can be used.	Slower because records may need to be checked one by one.
Best Use	Suitable for reports and applications requiring ordered data.	Suitable when frequent insertions are required.

Example

Suppose student records are:

Sequential File:

101 → 102 → 103 → 104

Heap File:

103 → 101 → 104 → 102

The sequential file maintains an order, while the heap file stores records wherever space is available.

Conclusion

Sequential file organization provides better **ordered retrieval**, but insertion and deletion can be costly. Heap file organization provides **fast insertion and simple deletion**, but searching is generally slower.

Q2. Clustered Indexing vs Non-Clustered Indexing

Introduction

Indexing is used to improve the speed of retrieving records from a database. **Clustered indexing** determines the physical order of records, whereas **non-clustered indexing** maintains a separate index structure containing references to the actual records.

Aspect	Clustered Indexing	Non-Clustered Indexing
Physical Order	Records are physically arranged according to the	Physical order of records is independent of the index.

	index key.	
Number of Indexes	Usually one clustered index can be created because data can have only one physical order.	Multiple non-clustered indexes can be created.
Storage	Data and index are closely associated.	Index is stored separately from the actual data.
Searching	Very efficient for ordered and range searches.	Efficient for searches on indexed columns but may require an additional lookup.
Range Queries	Generally very efficient.	May require multiple record lookups.
Example	Employee records physically arranged by Employee_ID.	Separate index created on Employee_Email.

Example

Consider an Employee table:

Employee_ID | Name | Department

If the records are physically stored in the order:

101 → 102 → 103 → 104

based on Employee_ID, the index on Employee_ID can be a **clustered index**.

If the table is physically stored differently but an index is created on Employee_Name, it is a **non-clustered index**.

Advantages

Clustered Indexing:

- Faster sequential access.
- Efficient for range queries.
- Good when records are frequently retrieved in sorted order.

Non-Clustered Indexing:

- Multiple indexes can be created.
- Useful for searching using different attributes.
- Does not change the physical arrangement of the table.

Conclusion

Clustered indexing is suitable when ordered and range-based retrieval is important. Non-clustered indexing is more flexible because indexes can be created on multiple columns.

Q3. Primary Indexing vs Secondary Indexing

Introduction

An index provides a faster path to locate records in a database. **Primary indexing** is generally associated with the primary or ordering key of a sorted file, while **secondary indexing** is created on a non-ordering field to provide additional access paths.

Aspect	Primary Indexing	Secondary Indexing
Index Key	Usually based on the primary/ordering key.	Based on a non-ordering attribute.
Data Ordering	Data file is generally ordered according to the index key.	Data file does not need to be ordered according to the indexed field.
Number	Usually one primary ordering index.	Multiple secondary indexes can be created.
Storage	Usually requires less additional storage.	Can require more storage, especially with multiple indexes.
Search Performance	Very efficient for primary-key searches.	Efficient for searches using other attributes.
Example	Student_ID.	Student_Name, Department, Email.

Example

Consider:

Student_ID Name Department

101	Asha	CSE
102	Ravi	ECE
103	Meena	CSE

An index on **Student_ID** can act as a primary index.

If an index is created on **Department**, it provides another access path and acts as a secondary index.

Analysis

Primary indexing is useful because the records are organized according to the primary or ordering key. Therefore, searching based on that key can be performed efficiently.

Secondary indexing improves access to fields that are not used to physically order the file. However, maintaining additional indexes requires extra storage and update operations.

Conclusion

Primary indexing provides efficient access through the main ordering key with relatively lower index overhead, while secondary indexing provides additional search options at the cost of extra storage and maintenance.

Q4. Static Hashing vs Dynamic Hashing

Introduction

Hashing maps a search key to a storage bucket using a hash function. **Static hashing** uses a fixed number of buckets, while **dynamic hashing** allows the number of buckets to change as the database grows.

Aspect	Static Hashing	Dynamic Hashing
Number of Buckets	Fixed.	Can increase or decrease.
Database Growth	Less suitable for continuously growing data.	Suitable for growing databases.
Overflow	Overflow chains may develop.	Buckets can be split to handle overflow.
Scalability	Limited.	Highly scalable.
Performance	May decrease when many records are added.	Can maintain better performance as data grows.
Complexity	Simple to implement.	More complex to implement.
Examples	Fixed hash table.	Linear hashing and extendible hashing.
Example		

Suppose a static hash table has **10 buckets**.

Even if thousands of records are inserted, the original bucket structure remains fixed. Many records may therefore fall into the same bucket and create overflow chains.

In dynamic hashing, when a bucket becomes full, the hashing structure can expand by splitting buckets.

Analysis

Static hashing is easy to implement and works well when the number of records is relatively stable. However, if the database grows significantly, collisions and overflow chains can increase.

Dynamic hashing adjusts the storage structure according to the amount of data. Therefore, it is more suitable for large and continuously growing databases.

Conclusion

Static hashing is simple and suitable for fixed-size data, whereas dynamic hashing provides better scalability and performance for growing databases.

Q5. B-Tree vs B+ Tree

Introduction

B-Tree and B+ Tree are balanced tree-based indexing structures used for efficient database searching. Both reduce disk accesses by maintaining a balanced structure, but their data storage and traversal methods are different.

Aspect	B-Tree	B+ Tree
Data Storage	Data can be stored in internal as well as leaf nodes.	Actual data entries are stored at leaf level.
Internal Nodes	Can contain keys and associated data.	Mainly contain keys and pointers to child nodes.
Leaf Nodes	Usually not linked sequentially.	Leaf nodes are linked.
Search	Search may finish at an internal node.	Search normally reaches the leaf level.
Range Queries	Less efficient compared to B+ Tree.	Very efficient because leaf nodes are linked.
Sequential Access	Less convenient.	Efficient through linked leaves.
Database Applications	Useful for general tree indexing.	Highly suitable for database and file-system indexing.
Example		

Suppose we want to retrieve all keys from **15 to 45**.

In a B+ Tree, after finding key 15, the system can follow the linked leaf nodes:

15 → 20 → 25 → 30 → 35 → 40 → 45

This makes range retrieval efficient.

Analysis

B-Trees can store data at different levels, so a search may finish before reaching the leaf. However, B+ Trees keep data entries at the leaf level and connect leaf nodes together.

Because of this structure, B+ Trees are especially useful for **range queries and sequential access**.

Conclusion

For large-scale database applications, B+ Tree indexing is generally more advantageous because of its efficient range searching, sequential traversal, and linked leaf nodes.

Q6. Linear Hashing vs Extendible Hashing

Introduction

Both linear hashing and extendible hashing are dynamic hashing techniques. They allow the hash structure to grow as more records are inserted, avoiding the limitations of fixed-size static hashing.

Aspect	Linear Hashing	Extendible Hashing
Bucket Growth	Buckets are split gradually in a predetermined sequence.	Bucket splits occur when overflow happens.
Directory	Does not require a directory of pointers.	Uses a directory to point to buckets.
Directory Doubling	Not required.	Directory may double when required.
Scalability	Good for gradual database growth.	Good for rapidly growing databases.
Memory Overhead	Lower because there is no large directory.	Higher due to directory storage.
Bucket Management	Controlled and gradual splitting.	Splits specific overflowing buckets.
Example		

In **linear hashing**, buckets are gradually split:

Bucket 0 → split → Bucket 1 → split → ...

In **extendible hashing**, a directory is used:

Directory
↓
Bucket 0
Bucket 1

When required, the directory can be doubled.

Analysis

Linear hashing avoids the need for a directory and grows gradually, making it useful when data increases continuously.

Extendible hashing can respond directly to bucket overflow and provides fast access through its directory. However, the directory requires additional memory.

Conclusion

Linear hashing provides gradual bucket growth with lower directory overhead, while extendible hashing provides flexible bucket management using a directory and is highly scalable.

Q7. Dense Indexing vs Sparse Indexing

Introduction

Dense and sparse indexing differ mainly in the number of entries maintained in the index. A **dense index** has an index entry for every record, whereas a **sparse index** has entries for only selected records, commonly one entry per block.

Aspect	Dense Indexing	Sparse Indexing
Entries	One entry for every record.	Fewer entries, usually one per block.
Memory Usage	High.	Low.
Search Speed	Faster because every record has an index entry.	Slightly slower because additional scanning may be needed.
Storage Requirement	More storage required.	Less storage required.
Maintenance	More entries need to be updated.	Easier to maintain.
Performance	Very fast direct access.	Good performance with lower storage.
Example		

Suppose there are five records:

101, 102, 103, 104, 105

A dense index may contain:

101 → 102 → 103 → 104 → 105

A sparse index may contain:

101 → 103 → 105

The sparse index contains fewer entries.

Analysis

Dense indexing provides faster searching because the index contains an entry for every record. However, it requires more memory and maintenance.

Sparse indexing saves storage because it contains fewer entries. After reaching the appropriate index entry, the system may need to scan some records to locate the exact record.

Conclusion

Dense indexing is preferred when faster access is more important, while sparse indexing is preferred when reducing index storage and memory usage is more important.

Q8. Indexed Sequential File Organization vs Direct File Organization

Introduction

Indexed Sequential File Organization and Direct File Organization provide different methods for accessing database records. Indexed sequential organization supports both ordered and indexed access, while direct organization provides direct access to a record using a calculated address or hash value.

Aspect	Indexed Sequential	Direct File Organization
Access	Sequential as well as indexed access.	Direct/random access.
Ordering	Records are maintained in an ordered manner.	Records need not be stored sequentially.
Exact Search	Efficient.	Very fast.
Range Queries	Efficient because records are ordered.	Less suitable.
Sequential Processing	Highly suitable.	Not its main strength.
Index	Requires index structure.	Usually uses direct address/hash calculation.

Advantages of Indexed Sequential

1. Supports sequential processing.
2. Provides faster indexed retrieval.
3. Suitable for range queries.
4. Records are maintained in an ordered manner.
5. Useful for applications requiring both sequential and random access.

Limitations

1. Index requires additional storage.
2. Index maintenance is required.
3. Insertions and deletions can be more complex.

Advantages of Direct File Organization

1. Very fast direct access.
2. Suitable for exact-match queries.
3. Avoids scanning the entire file.
4. Useful for applications requiring quick individual record access.

Limitation

Direct file organization is less suitable for ordered processing and range queries.

Example

For searching:

Student_ID = 105

Direct organization can calculate the location and access the record quickly.

For searching:

Student_ID from 100 to 150

Indexed sequential organization is more suitable because records are ordered and can be accessed sequentially.

Conclusion

Indexed sequential organization is better for ordered, sequential, and range access, while direct organization is better for fast exact-match retrieval.

Q9. Single-Level Indexing vs Multi-Level Indexing

Introduction

Indexing improves database retrieval by reducing the number of records that must be searched. In **single-level indexing**, only one index is maintained. In **multi-level indexing**, an additional index is created over the lower-level index, forming multiple levels.

Aspect	Single-Level Indexing	Multi-Level Indexing
Levels	One index level.	Multiple index levels.
Suitable Data Size	Small or moderate databases.	Large databases.
Searching	Searches one index.	Searches from top level to lower level.
Performance	Good for smaller indexes.	More efficient for large indexes.
Complexity	Simple.	More complex.
Storage	Lower structural complexity.	Requires additional index levels.

Single-Level Structure

Index



Data File

The index directly points to records or blocks.

Multi-Level Structure

Level 1 Index



Level 2 Index



Data File

The top-level index helps locate the required lower-level index, which then locates the required data.

Example

Suppose a database has a very large index containing thousands of entries. Searching the entire index may still require many operations.

In multi-level indexing, the first-level index helps identify the relevant second-level section, and the lower-level index then identifies the required data.

Analysis

Single-level indexing is simple and works well when the index itself is small. However, as the database grows, the index becomes large.

Multi-level indexing divides the index into smaller searchable sections. This reduces the search effort and is more suitable for large databases.

Conclusion

Single-level indexing is simple and suitable for smaller databases, while multi-level indexing provides better retrieval efficiency for large databases by organizing indexes into multiple levels.

Q10. Hashing Techniques vs Indexing Techniques for Exact-Match and Range Queries

Introduction

Hashing and indexing are important techniques for improving database retrieval performance. However, their effectiveness depends on the type of query. **Hashing is highly suitable for exact-match queries**, while **indexing, especially B+ Tree indexing, is more suitable for range queries**.

1. Exact-Match Queries

An exact-match query searches for one specific value.

Example:

Find Student_ID = 105

Hashing

A hash function directly maps 105 to a bucket.

105 → Hash Function → Bucket

Therefore, hashing can provide very fast direct access when the hash function distributes records effectively.

Indexing

An index can also locate Student_ID 105 efficiently, especially when using structures such as B-Trees or B+ Trees.

2. Range Queries

A range query searches for values within a specified range.

Example:

Find Student_ID between 100 and 150

Hashing

Hashing is not ideal for range queries because nearby key values do not necessarily produce nearby hash values.

For example:

100 → Bucket 4
101 → Bucket 8
102 → Bucket 1

The records are scattered across different buckets.

Indexing

An ordered index such as a B+ Tree is much more suitable.

Once the starting key is found, the system can traverse the ordered/linked entries to retrieve the remaining records.

Comparison

Feature	Hashing	Indexing
Exact Match	Excellent	Very good
Range Query	Poor	Excellent
Ordering	Does not maintain key order.	Can maintain key order.
Access Method	Direct bucket access.	Tree/index traversal.
Best Use	Exact-match searches.	Range and ordered searches.
Example	ID = 105	ID BETWEEN 100 AND 150
Analysis		

For exact-match queries, hashing can provide very fast access because the search key is converted directly into a bucket address. Its performance depends on the quality of the hash function and collision handling.

For range queries, indexing is more efficient because indexed structures such as B+ Trees maintain keys in an ordered form. Therefore, multiple consecutive values can be retrieved efficiently.

Conclusion

Hashing is generally the better choice for exact-match queries, while indexing is better for range and ordered queries. Therefore, database designers should select hashing or indexing based on the expected query pattern and retrieval requirements.